Q13. Packet-capture protocol analysis

Original question: Analyze a network packet capture. Identify TCP, UDP, DNS, and ICMP traffic wherever available. Record the source and destination addresses, protocol types, and observations about the captured packets.

Aim

Identify TCP, UDP, DNS and ICMP traffic actually present in a supplied capture, and record endpoint and packet observations.

Requirements

PCAP/PCAPNG, Wireshark and optionally TShark. Reading an existing file does not require capture privilege. References: [R12], [R13], [R14].

Procedure

Step 1 - Preserve and summarize the capture

```
mkdir -p "$HOME/EHF_Exam/Q13"
cd "$HOME/EHF_Exam/Q13"
PCAP='<PCAP_FILE>'
sha256sum "$PCAP" | tee capture_before.sha256
capinfos "$PCAP" | tee capture_info.txt
```

`capinfos` reports capture properties such as packet count, time range and encapsulation. Record timezone/time-display settings. An incomplete/truncated capture limits the available payload and handshake analysis.

Step 2 - Open and inspect in Wireshark

1. Open Wireshark, select File > Open and choose the capture. Do not start a live capture accidentally.
2. Open Statistics > Protocol Hierarchy for a protocol overview. Open Statistics > Endpoints/Conversations to review actual communicating addresses.
3. Apply each display filter below in the filter bar and press Enter. Record the displayed count and at least one frame number for each present protocol. If none match, record not observed.

```
tcp
udp
dns
icmp || icmpv6
```

4. Select a matching packet. Expand Ethernet (if present), Internet Protocol, TCP/UDP and DNS/ICMP sections. Record source/destination addresses, ports where applicable, flags or query/type details, timestamp and frame number.
5. Save observations or export the displayed packet dissection through File > Export Packet Dissections. Keep the original capture unchanged. DNS can run over UDP or TCP, so protocol counts can overlap; do not add them as disjoint totals.

Step 3 - Export a concise terminal summary

```
tshark -n -r "$PCAP" -Y 'tcp || udp || dns || icmp || icmpv6' \
  -T fields -E header=y -E separator=, -E quote=d \
  -e frame.number -e frame.time_epoch \
  -e ip.src -e ipv6.src -e ip.dst -e ipv6.dst \
  -e _ws.col.Protocol -e tcp.srcport -e tcp.dstport \
  -e udp.srcport -e udp.dstport > packet_summary.csv
sed -n '1,25p' packet_summary.csv
sha256sum -c capture_before.sha256
```

`-r` reads a capture; `-Y` applies a display filter; `-T fields` selects fields; `-e` adds a field. `-n` avoids name resolution. Empty IPv4 fields in an IPv6 packet are normal. The protocol column is a dissection summary; inspect layers for precise protocol details.

Expected output - illustrative observation

```
Frame 12 | 192.0.2.10 -> 192.0.2.53 | DNS over UDP
Meaning: a name query is visible; inspect qname/type and reply.
Frame 18 | 192.0.2.10 -> 192.0.2.20 | TCP SYN
Meaning: a connection attempt, not proof it completed.
```

Frame numbers and documentation addresses are synthetic. Use only frames present in your capture.

Interpretation, observation and result

Make a table: frame/time | source | destination | transport/application | ports/flags | observation. TCP is connection-oriented; UDP lacks a transport handshake; DNS resolves names; ICMP carries control/error or echo traffic. An ICMP error may mean unreachable delivery, not an attack. Encrypted HTTPS usually hides application contents; DNS over HTTPS/TLS may not appear under dns. The result identifies protocols and concrete packet evidence, including any absent categories and capture limitations.

Common errors

- Red filter bar: check spelling/operators and use a display filter, not a capture-filter expression.
- No DNS: confirm capture location, caching and encrypted DNS; do not invent queries.
- Permission denied on opening: obtain file read access; launching the entire GUI as root is unnecessary.

Viva questions

- Are capture and display filters identical? No; they use different syntax and act at different stages.
- Can DNS use TCP? Yes; it is not limited to UDP.
- Why record frame numbers? They allow the examiner to locate the exact packet.
- Does a SYN prove a session exists? No; inspect the matching response and acknowledgement.